

Phase 2: Speech & Audio Processing

Presentation Confidence Detector

Project Blueprint

Phase 2: Speech & Audio Processing

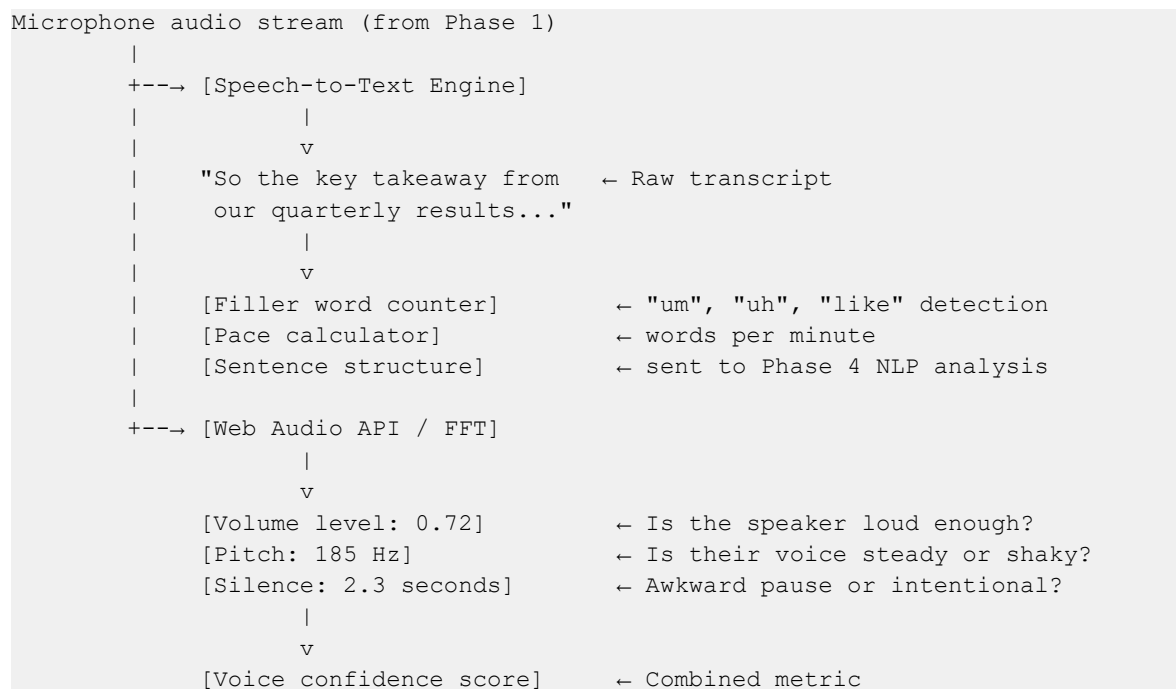
What Is This Phase?

This is the **biggest new territory** in the Confidence Detector. ExamGuard never touched audio — it was purely visual. Now you need to do two things with the microphone feed:

1. **Transcribe speech to text** (what is the person SAYING?)
2. **Analyze voice characteristics** (HOW are they saying it — loud? shaky? monotone?)

These are two completely different pipelines running on the same audio stream. Speech-to-Text converts sound waves into words. Audio analysis extracts raw numbers like pitch, volume, and frequency — things that reveal nervousness, confidence, and energy.

WHY This Phase Matters for the Confidence Detector



A confident speaker has steady pitch, adequate volume, minimal filler words, and intentional pauses. A nervous speaker has rising pitch, quiet voice, excessive "um"s, and either no pauses or too-long silences. **This phase gives you the data to tell the difference.**

Skills to Learn

1. Speech-to-Text Options — Converting Speech to Text

What is it?	Speech-to-Text (STT) engines listen to audio and produce text. There are multiple options with very different tradeoffs — some run in the browser for free, others run on a server with higher accuracy, others cost money per minute of audio.
-------------	---

WHY it matters	The Confidence Detector needs a live transcript to count filler words, measure speaking pace, and feed text to the NLP pipeline in Phase 4. The STT engine you choose determines accuracy, latency, cost, and whether you need a server.
-----------------------	--

Options to understand:

Engine	Runs Where	Cost	Accuracy	Real-time?	Best For
Web Speech API	Browser (best for Chrome/Edge prototypes)	Free	Medium	Yes	Fast MVP / prototyping
Whisper (OpenAI)	Server (Python)	Free (self-hosted)	Very High	No (batch)	Post-session analysis
Vosk	Browser or server	Free	Good	Yes	Offline / privacy-first
Deepgram	Cloud API	Paid	Very High	Yes	Production real-time

What to learn:

You create a `SpeechRecognition` instance and configure it for continuous listening with interim (partial) results enabled. When the `onresult` event fires, you loop through the results array starting from `resultIndex` to build the transcript string. Calling `recognition.start()` begins listening through the microphone.

Confidence Detector connection: For the browser-first MVP, use the **Web Speech API** only if you want the fastest prototype in Chrome/Edge and can accept browser-specific behavior. If you need predictable offline or cross-browser STT, plan to swap to Vosk or a server-side engine later. The important thing is getting a transcript flowing so you can count filler words and measure pace.

2. Interim vs Final Results — Real-Time Transcription Handling

What is it?	STT engines produce two types of results: interim (the engine's current best guess while the person is still talking) and final (the confirmed text after the engine is confident). Interim results change rapidly; final results are locked in.
WHY it matters	If you count filler words on interim results, you will double-count everything because interim results keep changing. You need to count on FINAL results only. But you display INTERIM results so the user sees live text.

What to learn:

In the `onresult` handler, you check each result's `isFinal` property to separate confirmed text from in-progress guesses. Final results are used for analysis (filler word counting, pace measurement) because they will not change, while interim results are displayed in a lighter style so the user sees text appearing as they speak. This prevents double-counting metrics that would happen if you analyzed the constantly-changing interim text.

Confidence Detector connection: The live transcript panel shows interim results (so the user sees text appearing as they speak), but all analysis (filler word counting, pace measurement) runs on final results only. This prevents the metrics from flickering and double-counting.

3. Auto-Restart Pattern — Handling STT Engine Timeouts

What is it?	The Web Speech API has an annoying behavior: it stops listening after a period of silence or after a timeout (varies by browser, usually 30-60 seconds). Your app must detect this and automatically restart the engine without the user noticing.
WHY it matters	A presentation practice session can last 5-30 minutes. If the STT engine dies after 60 seconds and you do not restart it, you lose the entire transcript from that point forward. The user thinks the app is still listening, but it is not.

What to learn:

You track a boolean `isListening` flag and create a `startListening` function that guards against double-starts. In the `onend` handler, you automatically restart recognition if the session is still active, since the Web Speech API silently stops after periods of silence. In the `onerror` handler, you check the error type: `no-speech` triggers a quiet restart, while `audio-capture` and `not-allowed` errors show user-facing messages about microphone problems.

Confidence Detector connection: This is a reliability pattern. Without it, your app silently breaks after the first minute. With it, the STT engine auto-recovers and the user never notices. This is the kind of detail that separates a demo from a real tool.

4. Web Audio API — Analyzing Raw Audio

What is it?	The Web Audio API gives you access to the raw audio data from the microphone — not as text, but as numbers representing the sound wave. You can measure how loud the sound is, what frequencies are present, and how the audio changes over time.
WHY it matters	Speech-to-Text tells you WHAT the person said.

	The Web Audio API tells you HOW they said it. Was their voice loud or quiet? Steady or shaking? High-pitched (nervous) or low-pitched (calm)? These are the voice metrics that indicate confidence.
--	---

What to learn:

You create an `AudioContext`, connect the microphone stream as a source, and pipe it through an `AnalyserNode` with an FFT size of 2048 for frequency resolution. In a `requestAnimationFrame` loop, you pull raw audio data into a typed array using either `getByteTimeDomainData` (waveform) or `getByteFrequencyData` (frequency spectrum), giving you numbers from 0-255 that represent the audio signal each frame. These numbers are then processed to extract volume, pitch, and other voice characteristics.

Confidence Detector connection: The `analyser` node is your audio equivalent of OpenCV's frame reading. Every animation frame, you pull out raw audio numbers and analyze them — just like every video frame gets pulled from the camera and analyzed for faces. The Web Audio API is the audio pipeline foundation.

5. FFT (Fast Fourier Transform) Basics — Extracting Pitch and Frequency

What is it?	FFT is a mathematical algorithm that converts a sound wave (amplitude over time) into a frequency spectrum (which frequencies are present and how strong each is). The Web Audio API's <code>AnalyserNode</code> does FFT for you — you just read the results.
WHY it matters	Human voice pitch is a frequency. A calm, confident voice sits around 100-150 Hz for males, 180-250 Hz for females. When someone gets nervous, their pitch RISES. FFT lets you measure this. You do not need to understand the math — you need to understand what the output means.

What to learn:

The `AnalyserNode` internally performs FFT and divides the audio into frequency bins (with `fftSize` / 2 bins total). Each bin covers a range of frequencies calculated as `sampleRate` / `fftSize` (about 21.5 Hz per bin at standard 44100 Hz sample rate). To find the dominant pitch, you loop through all bins to find the one with the highest energy value, then multiply that bin's index by the bin width to get the frequency in Hz — for example, bin 10 would correspond to roughly 215 Hz, which is in the female voice range.

Confidence Detector connection: You do not need to be an FFT expert. You need to know that the `AnalyserNode` gives you frequency bins, that voice pitch lives in the 80-400 Hz range, and how to

find the dominant frequency. That is enough to track whether someone's pitch rises (nervousness) or stays steady (confidence).

6. Volume and Pitch Tracking — Measuring Voice Characteristics Over Time

What is it?	Instead of a single volume or pitch reading, you track these values OVER TIME — building a history that shows patterns. A confident speaker's volume and pitch are relatively stable. A nervous speaker's volume drops and pitch jumps around.
WHY it matters	A single snapshot tells you nothing. "Volume is 0.6" means nothing by itself. But "volume started at 0.7 and dropped to 0.3 over 2 minutes" tells a story — the speaker is losing confidence. Tracking over time is what makes the analysis meaningful.

What to learn:

You build a `VoiceTracker` class that stores timestamped volume and pitch readings in arrays, capping the history at about 10 seconds to avoid memory issues. It provides methods to compute the rolling average volume, the volume variation (standard deviation over a recent window, where high variation means an unstable voice), and the volume trend (comparing the average of early readings to recent readings to detect whether the speaker is getting louder or fading). These metrics reveal patterns over time rather than just single-moment snapshots.

Confidence Detector connection: The `VoiceTracker` class is a core component. It feeds into the confidence score: stable volume + steady pitch + adequate loudness = confident voice. The trend data also powers the post-session report: "Your volume dropped in the last 2 minutes — you may have lost energy at the end."

7. Silence Detection — Detecting Pauses in Speech

What is it?	Silence detection measures when the speaker STOPS talking. You set a volume threshold, and when the audio level stays below it for a certain duration, you classify it as a pause. Short pauses are normal and even good; long pauses indicate uncertainty or loss of train of thought.
WHY it matters	Pauses are a key confidence signal. Skilled speakers use intentional pauses for emphasis. Nervous speakers either rush through with no pauses at all, or freeze with long awkward silences. The Confidence Detector needs to tell the difference.

What to learn:

You build a `SilenceDetector` class that tracks whether the current volume is below a configurable threshold. When volume drops below the threshold, it starts timing the silence; when volume rises back above, it records the pause with its duration and classifies it as "normal" (300ms+) or "long/awkward" (3 seconds+). A `getStats` method returns the total pause count, how many were normal vs long, and the average pause duration, which helps distinguish intentional dramatic pauses from nervous freezes.

Confidence Detector connection: Silence detection feeds directly into the confidence score AND the real-time feedback. If the user pauses for 5 seconds, the UI can gently nudge: "Long pause detected." The post-session report can say: "You had 3 pauses longer than 3 seconds, all in the second half — consider rehearsing your closing."

WHY Vosk for Production?

The MVP uses Web Speech API because it is the fastest path to a working prototype. But when you need to support multiple users, work offline, or run in any browser, you need a real STT engine. Here is the honest comparison:

Factor	Web Speech API	Vosk	Whisper (OpenAI)
Cost	Free	Free (open source)	Free (self-hosted) or paid (API)
Browser support	Chrome/Edge ONLY	Any browser (via WASM) or server	Server only
Internet required?	Usually yes (Chrome sends audio to Google servers)	No — fully offline	No if self-hosted, yes if API
Real-time?	Yes — streaming results	Yes — streaming results	No — batch processing. You send a file, wait, get text back.
Accuracy	Medium (good for clear English)	Good (accent-specific models available)	Best available (especially Whisper large)
Session timeout	30-60 seconds, then silently stops	No timeout — runs as long as you want	N/A (batch)
RAM per user	Browser handles it	~200 MB per concurrent user (server mode)	~1-2 GB for small model, 4-10 GB for large
GPU needed?	No (runs in cloud)	No — CPU is fine	Small model: no. Large model: yes, strongly recommended.
Privacy	Audio may be sent to Google servers	Audio stays on device/your server	Audio stays on your server
Multiple users	Each user's browser handles it	Server: 200 MB RAM x N users. 10 users = 2	Server: needs GPU. Expensive to scale.

		GB.	
--	--	-----	--

For many concurrent users, Vosk wins. Here is the reasoning:

- Web Speech API cannot scale — it is browser-only, Chrome-only, has timeouts, and you cannot control it server-side.
- Whisper is the most accurate but needs a GPU and processes in batch (not real-time). Great for post-session analysis, but you cannot show live transcripts with it.
- Vosk runs real-time on CPU, works offline, has no timeouts, and each user costs about 200 MB RAM. A server with 8 GB RAM can handle 30+ concurrent users. That is practical for a classroom or workshop demo.

The migration path: Start with Web Speech API (zero setup, instant prototype), then swap to Vosk when you need cross-browser support or multiple users. The transcript output format is the same — your filler detection and NLP pipeline do not need to change.

Complete Filler Word List

Every filler word the Confidence Detector detects, organized by type, with WHY each one counts:

Pure Verbal Pauses

These are sounds, not words. The speaker's brain is stalling while formulating the next thought.

Filler	What It Sounds Like	WHY It Counts
um	"The results were, um, positive"	Classic thinking pause. Universal across English speakers. The most recognized filler.
uh	"We should, uh, consider..."	Shorter version of "um." Same function — buying time to think.
ah	"Ah, that is a good point"	Often used before responding to a question. Signals the speaker was caught off guard.
er	"The number is, er, around fifty"	More common in British English. Same as "uh" — a vocal placeholder.
hmm	"Hmm, let me think about that"	Can be intentional (showing you are thinking) or a stall. Context matters but we count it.

Filler Words Disguised as Real Words

These ARE real English words, but in presentations they are used as fillers 85% of the time.

Filler	Filler Usage	Legitimate Usage	WHY We Count It
--------	--------------	------------------	-----------------

like	"It was, like, really important"	"I like pizza"	Filler 85% of the time in spoken presentations. The ~15% false positive rate is acceptable.
so	"So, basically, what happened was..."	"So the total is 500" (transitional)	Often used to start sentences without purpose. We count it when it appears as a sentence opener.
right	"And then, right, we moved on"	"That is right" (confirmation)	Used as a verbal tick seeking agreement. Very common in casual presentations.
well	"Well, I would say that..."	"The system works well"	Sentence-opening filler. Delays getting to the point.
okay	"Okay, okay, so the next slide..."	"The results are okay"	Verbal crutch, often repeated. Signals nervousness or uncertainty about transitions.

Verbal Padding

These words add zero meaning. They inflate sentences to sound more substantial but actually signal the speaker is not sure of what they are saying.

Filler	Example	WHY It Counts
basically	"Basically, the system works by..."	If you need to say "basically," the explanation is already basic. It adds nothing.
actually	"We actually found that..."	Implies surprise at your own findings. Undermines your authority.
literally	"It literally changed everything"	Almost never used literally. Verbal emphasis that reveals insecurity about the claim.

Conversation Fillers

These come from casual conversation habits and leak into presentations.

Filler	Example	WHY It Counts
you know	"The data, you know, showed growth"	Seeking validation from the audience mid-sentence. Confident speakers state facts without checking.
I mean	"I mean, the results are clear"	Self-correction without an

		actual correction. Verbal hesitation.
--	--	---------------------------------------

Softeners

These weaken statements by hedging. They overlap with the hedge phrase list in Phase 4 but are also counted as fillers.

Filler	Example	WHY It Counts
sort of	"It sort of worked"	Did it work or not? Softeners make definitive statements wishy-washy.
kind of	"We kind of achieved the goal"	Same as "sort of." Undercuts the speaker's own conclusions.

Total: 18 filler words/phrases. In the MVP, we count all of them equally. In v2, you could weight pure verbal pauses heavier than softeners, since "um" is more distracting to an audience than "basically."

The Accent Challenge

Honest truth: **speech-to-text accuracy drops significantly with non-standard accents.** This is a known limitation of all STT systems, not just ours.

Accent Type	Web Speech API Accuracy	Vosk Accuracy	Impact on Confidence Detector
Standard American English	~90-95%	~85-90%	Baseline — system works as designed
Standard British English	~85-90%	~80-85%	Minor impact. Most fillers still detected.
Indian English	~70-80%	~75-85% (with Hindi model)	Filler detection degrades. "Actually" may be misheard.
South Asian accents broadly	~65-80%	~70-80% (accent-specific models)	Noticeable impact. False filler counts increase.
Strong regional accents (any)	~60-75%	~65-80%	Significant impact. WPM calculation becomes unreliable.

What this means for the Confidence Detector:

- If the STT mishears "and" as "um," the filler counter goes up when it should not. False positives hurt trust.
- If the STT misses a real "um," the filler counter is too low. False negatives make the tool less useful.

- WPM calculation becomes unreliable because missed words mean lower word count per time window.

What we can do about it:

3. **Vosk has accent-specific models.** There are models trained on Indian English, German English, etc. Using the right model improves accuracy by 10-15%.
4. **Set a minimum confidence threshold.** Both Web Speech API and Vosk return a confidence score per word. Only count fillers where the STT is 70%+ confident in the transcription.
5. **Be transparent with users.** Show the raw transcript so the user can see when the STT is making mistakes. If they see "um" in the transcript where they said "and," they know to discount the filler count.
6. **This is an active area of improvement, not a solved problem.** Even Google and Apple struggle with accent diversity. Acknowledge the limitation, mitigate where possible, and improve over time.

Interim vs Final: Real Example

This is confusing until you see it in action. Here is exactly what happens when someone says the sentence "I think the quarterly results were strong":

Time	What the Person Has Said So Far	Interim Result (changing)	Final Result (locked)
0.0s	"I..."	"I"	(nothing yet)
0.3s	"I think..."	"I think"	(nothing yet)
0.6s	"I think the..."	"I think the"	(nothing yet)
0.9s	"I think the quart..."	"I think the court"	(nothing yet)
1.1s	"I think the quarterly..."	"I think the quarterly"	(nothing yet)
1.4s	"I think the quarterly results..."	"I think the quarterly results"	(nothing yet)
1.8s	"I think the quarterly results were..."	"I think the quarterly results were"	(nothing yet)
2.1s	"I think the quarterly results were strong."	"I think the quarterly results were strong"	(nothing yet)
2.5s	(pause — speaker stops)	(cleared)	"I think the quarterly results were strong"

Key observations:

7. **At 0.9 seconds, "quarterly" was heard as "court."** The speaker was mid-word. The STT engine's best guess for "quart-" was "court." If you ran filler detection on this interim result, you would analyze a word that does not exist in the actual speech. This is why you never analyze interim results.
8. **The interim result updated 8 times.** If you counted fillers on every interim update, you would count "I think" as a hedge phrase 8 times instead of once. This is the double-counting problem.

9. **The final result arrived only ONCE, at 2.5 seconds.** After the speaker paused, the STT engine locked in its best transcription. This is the one and only time you should run analysis.
10. **You DO display interim results in the UI** — the user sees text appearing in real-time as they speak. This feels responsive and confirms the system is listening. But the analysis pipeline (filler counting, hedge detection, pace calculation) only runs when `isFinal === true`.

The rule is simple: Show interim to the user. Analyze only final. If you remember one thing from this section, remember this.

Skill Summary Table

Skill	What It Does	Confidence Detector Use	New or Reused?
Speech-to-Text (STT)	Convert speech to words	Live transcript + filler word counting	NEW
Interim vs Final results	Handle partial transcription	Accurate counting without double-counting	NEW
Auto-restart pattern	Keep STT alive during long sessions	Reliability for 5-30 minute sessions	NEW
Web Audio API	Access raw audio numbers	Foundation for all voice analysis	NEW
FFT basics	Extract frequencies from audio	Measure voice pitch	NEW
Volume + pitch tracking	Track voice metrics over time	Confidence trend analysis	NEW
Silence detection	Detect pauses in speech	Distinguish intentional vs awkward pauses	NEW

Every single skill in this phase is NEW. ExamGuard had zero audio processing. This is your biggest learning investment.

After This Phase

After this phase, you can transcribe speech in real-time AND analyze voice characteristics.

You will have a working speech-to-text pipeline, filler word detection, speaking pace measurement, volume/pitch tracking, and silence detection. Combined, these produce a "voice confidence score" — one of the three major inputs to the overall Confidence Detector (the others being face/expression from Phase 3 and text analysis from Phase 4).

Resources

Official Documentation

Resource	Link	What You Get
----------	------	--------------

MDN: Web Speech API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API	SpeechRecognition reference
MDN: Web Audio API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API	AudioContext, AnalyserNode
MDN: AnalyserNode	https://developer.mozilla.org/en-US/docs/Web/API/AnalyserNode	FFT data, frequency/time domain
OpenAI Whisper (GitHub)	https://github.com/openai/whisper	High-accuracy STT for post-processing
Vosk (GitHub)	https://github.com/alphacep/vosk-api	Offline STT, runs in browser via WASM

Video Tutorials

Resource	Link	What You Get
The Coding Train: Speech Recognition	https://www.youtube.com/watch?v=q_bXBcmfTJM	Web Speech API walkthrough
Traversy Media: Web Speech API	https://www.youtube.com/watch?v=0mJCOA72Fnw	Practical STT demo
The Coding Train: Sound Visualization	https://www.youtube.com/watch?v=2O3nm0Nvbi4	Web Audio API + frequency visualization
Fireship: Web Audio API in 100 Seconds	https://www.youtube.com/watch?v=p0Fv9CDs1OA	Quick overview of audio processing
3Blue1Brown: Fourier Transform	https://www.youtube.com/watch?v=spUNpyF58BY	Best visual explanation of FFT concepts

Articles and Guides

Resource	Link	What You Get
Web Audio API Guide (MDN)	https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Using_Web_Audio_API	Step-by-step Web Audio tutorial
Pitch detection algorithms	https://github.com/cwilso/PitchDetect	Working pitch detection in JavaScript
Speech recognition best practices	https://developer.chrome.com/blog/voice-driven-web-apps-introduction-to-the-web-speech-api/	Chrome team's guide to Web Speech API

Key Takeaway

Audio processing is entirely new ground. In ExamGuard, you only had eyes (camera). The Confidence Detector adds ears (microphone). The speech-to-text pipeline gives you WHAT the speaker says. The audio analysis pipeline gives you HOW they say it. Together, they produce the audio side of the confidence signal; the other scored sources are face analysis in Phase 3 and transcript scoring in Phase 4. Take extra time on this phase because none of it carries over from ExamGuard.